

Ecuaciones diferenciales ordinarias

Tema 4. Derivación numérica y ecuaciones diferenciales ordinarias

Cálculo Numérico
Diego Rojo



Motivación

Aplicaciones de las EDOs

- Las **EDOs** aparecen en numerosos modelos de ciencia e ingeniería:
 - **Mecánica:** segunda ley de Newton $m\ddot{\mathbf{r}} = \mathbf{F}(t, \mathbf{r}, \dot{\mathbf{r}})$.
 - **Circuitos eléctricos:** $L\dot{i} + Ri + \frac{1}{C} \int i dt = V(t)$.
 - **Dinámica de poblaciones:** $\dot{N} = rN(1 - N/K)$ (ecuación logística).
 - Cinética química, epidemiología, finanzas cuantitativas, ...
- En general no admiten **solución analítica cerrada**: se impone el recurso a métodos numéricos.
- **Idea central:** emplear las diferencias finitas del bloque anterior para transformar la EDO en un **sistema algebraico**.

Ejemplo: proyectil con rozamiento del aire

Movimiento 2D bajo gravedad y fuerza de resistencia al aire $\mathbf{F}_d = -k m |\mathbf{v}| \mathbf{v}$:

$$\ddot{x} = -k \dot{x} \sqrt{\dot{x}^2 + \dot{y}^2}, \quad \ddot{y} = -g - k \dot{y} \sqrt{\dot{x}^2 + \dot{y}^2}.$$

- Sin arrastre ($k = 0$): trayectoria parabólica, solución analítica elemental.
- Con arrastre: el término $|\mathbf{v}| \mathbf{v}$ es **no lineal**; la ecuación no admite solución en forma cerrada.
- Datos iniciales: posición $(x(0), y(0))$ y velocidad $(\dot{x}(0), \dot{y}(0))$.
- La integración numérica es imprescindible en simuladores físicos realistas.

Estrategia: del problema continuo al discreto

- Una EDO relaciona una función desconocida con sus derivadas.
- **Obstáculo:** la derivada es un operador continuo; el ordenador opera con valores discretos.
- **Estrategia:**
 - i. Discretizar el intervalo de interés mediante una **mall**a $t_0 < t_1 < \dots < t_N$.
 - ii. Sustituir las derivadas por **diferencias finitas**.
 - iii. Obtener una **fórmula de recurrencia** que proporcione y_{i+1} a partir de valores previos.
- Esta estrategia da lugar a los métodos que veremos a continuación: Euler (explícito e implícito) y Heun.

Problema de Valor Inicial

Formulación general

Un **problema de valor inicial** (PVI) de primer orden se escribe como:

$$\frac{d}{dt}y(t) = f(t, y(t)), \quad a \leq t \leq b, \quad y(a) = y_0.$$

- La variable independiente se suele denotar t (tiempo).
- y_0 es el **estado inicial** del sistema.
- La función $f(t, y)$ de dos variables **define** la dinámica.
- Queremos aproximar $y(t)$ en el intervalo $[a, b]$.
- Si f no depende explícitamente de t (p. ej. $f(t, y) = y$), la EDO se llama **autónoma**.

Forma integral de la EDO

Integrando la EDO de a a t y usando el teorema fundamental del cálculo:

$$y(t) = y_0 + \int_a^t f(s, y(s)) ds.$$

- **Sin y en el integrando** ($f = f(t)$): es literalmente calcular una integral definida.
- **Caso general:** y aparece **dentro** del integrando $\Rightarrow$ el problema es más rico que una simple cuadratura.
- Esta visión será útil en el tema siguiente (**integración numérica**): permitirá derivar métodos para EDOs aproximando la integral con reglas tipo trapecio, Simpson, ...

Buen planteamiento

- Un PVI físicamente significativo requiere **existencia y unicidad** de solución (*principio de determinismo*).
- Si f es discontinua o el problema admite varias soluciones, cualquier predicción numérica pierde sentido.
- **Condición estándar:** f continua y **Lipschitziana en y** : existe $L \geq 0$ tal que
$$|f(t, y_1) - f(t, y_2)| \leq L |y_1 - y_2|, \quad \forall t \in [a, b], \quad y_1, y_2 \in \mathbb{R}.$$
- **Condición suficiente:** si f tiene derivada parcial acotada $|\partial f / \partial y| \leq M$, entonces f es Lipschitz con constante $L = M$ (consecuencia del teorema del valor medio).

Existencia y unicidad de la solución

Teorema (Picard-Lindelöf)

Sea $f(t, y)$ continua en $[a, b] \times \mathbb{R}$ y que satisface una condición de Lipschitz en la variable y con constante $L \geq 0$, uniformemente en t . Entonces, para cualquier valor inicial $y_0 \in \mathbb{R}$, el problema de valor inicial

$$y'(t) = f(t, y(t)), \quad a \leq t \leq b, \quad y(a) = y_0$$

tiene una **única solución** $y(t) \in C^1[a, b]$.

Demostración: Computational Mathematics (Dimitrios Mitsotakis) — Sección 8.2.1

- En lo sucesivo supondremos f suficientemente regular para garantizar existencia y unicidad.
- **Objetivo:** aproximar numéricamente $y(t)$.

Discretización del intervalo

- Elegimos un entero N y construimos una **mall**a uniforme:

$$t_i = a + i h, \quad i = 0, 1, \dots, N, \quad h = \frac{b - a}{N}.$$

- h se llama **paso** (*stepsize* o *timestep*).
- Los t_i son los **nod**os o *grid points*.
- Buscaremos aproximaciones $y_i \approx y(t_i)$ para $i = 1, 2, \dots, N$.
- Entre nodos podemos interpolar si necesitamos el valor en otros instantes.
- La mall
a **no tiene por qué ser uniforme** (métodos con paso adaptativo), pero lo asumiremos así por simplicidad.

Método de Euler (explícito)

Discretización de la derivada

- Evaluamos la EDO en el nodo t_i :

$$y'(t_i) = f(t_i, y(t_i)).$$

- Sustituimos $y'(t_i)$ por la **diferencia hacia adelante** del bloque anterior:

$$y'(t_i) \approx \frac{y(t_{i+1}) - y(t_i)}{h}.$$

- Denotando $y_i \approx y(t_i)$, se obtiene la fórmula de recurrencia:

$$\frac{y_{i+1} - y_i}{h} = f(t_i, y_i).$$

El método de Euler (explícito)

Despejando y_{i+1} :

$$y_{i+1} = y_i + h f(t_i, y_i), \quad i = 0, 1, \dots, N - 1, \quad y_0 = y(a)$$

- Dada y_i , obtenemos y_{i+1} **directamente** (sin resolver ninguna ecuación): es un método **explícito**.
- Conocido también como **Euler hacia adelante** (*forward Euler*).
- Es el método numérico **más simple** para EDOs.

Algoritmo

Entrada: f, a, b, N, y_0

Salida: $\{(t_i, y_i)\}_{i=0}^N$

$h = (b - a) / N$

$t_0 = a$

$y_0 = y_0$

para $i = 0, 1, \dots, N-1$:

$t_{i+1} = t_i + h$

$y_{i+1} = y_i + h * f(t_i, y_i)$

devolver $\{(t_i, y_i)\}$

- **Coste:** N evaluaciones de f . Lineal en N .
- **Memoria:** $O(N)$ para guardar la trayectoria (si sólo necesitamos y_N , $O(1)$).

Implementación en Python

```
import numpy as np

def euler(t, f, y0):
    n = len(t)
    y = np.zeros(n)
    h = (t[-1] - t[0]) / (n - 1)
    y[0] = y0
    for i in range(n - 1):
        y[i+1] = y[i] + h * f(t[i], y[i])
    return y
```

- `t` es el vector de nodos; `f(t, y)` la función de la EDO; `y0` el valor inicial.
- Devuelve el vector `y` con las aproximaciones en los nodos.

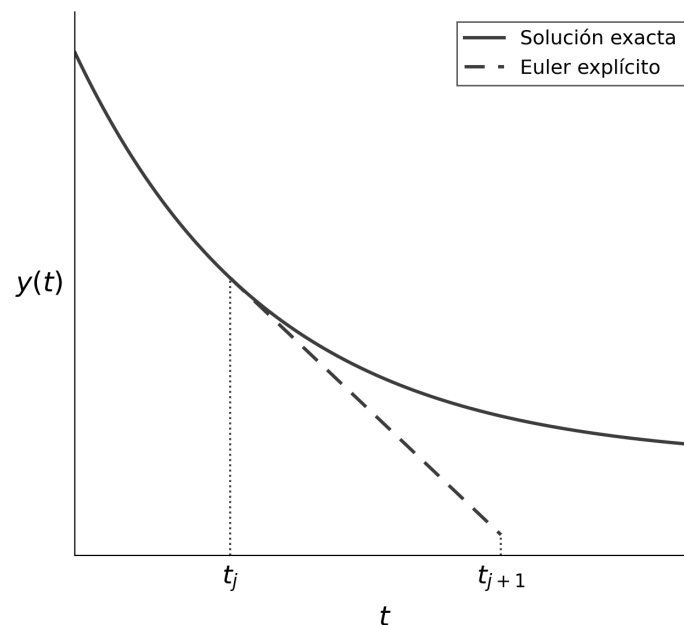
Ejemplo: $y' = y, y(0) = 1$

- EDO autónoma con solución exacta $y(t) = e^t$.
- Tomamos $[0, 2]$ y $h = 0.5$ ($N = 4$). Euler da $y_{i+1} = 1.5 y_i$.

t_i	y_i (Euler)	$y(t_i) = e^{t_i}$
0.0	1.0000	1.0000
0.5	1.5000	1.6487
1.0	2.2500	2.7183
1.5	3.3750	4.4817
2.0	5.0625	7.3891

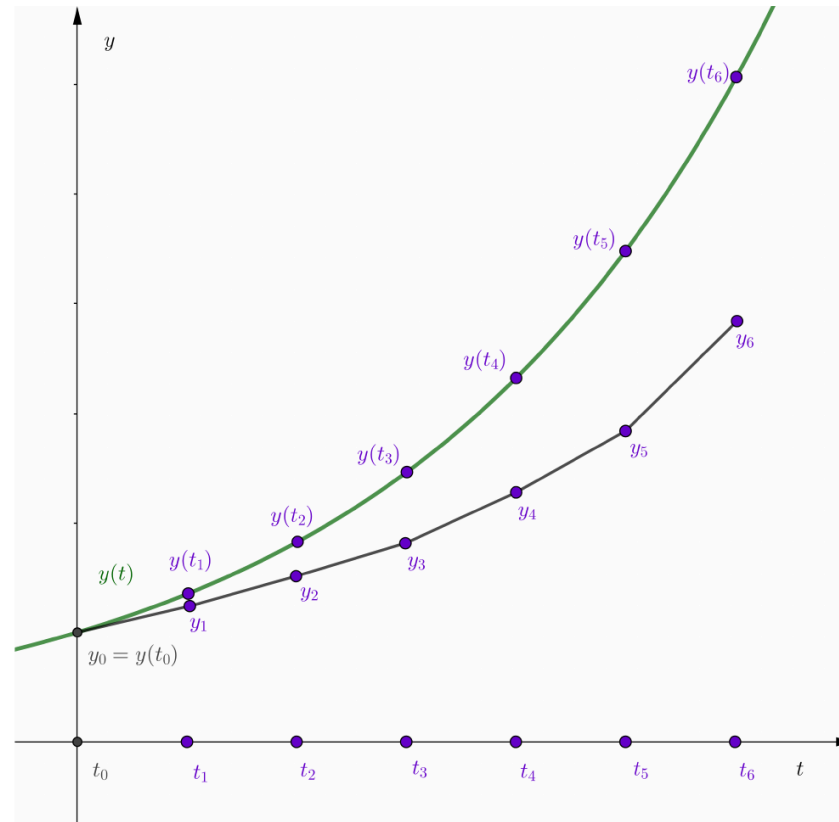
- Error considerable con $h = 0.5$. Con $h = 0.1$ ($N = 20$): $y_{20} \approx 6.73$. Con $N = 200$ el error es ya muy pequeño.

Interpretación geométrica: un paso



- Al inicio, (t_0, y_0) está **sobre** la solución exacta: el primer paso sigue la tangente real.
- Pero a partir del segundo paso, partimos de (t_i, y_i) con $y_i \neq y(t_i)$: la tangente que tomamos ya **no es** la tangente a la solución exacta.

Interpretación geométrica: acumulación



Iterando: cada y_{i+1} está sobre la tangente a la trayectoria que pasa por (t_i, y_i) , no sobre la tangente a la solución exacta. El error **crece con t** .

Análisis del Error

Error local y error global

En los métodos para EDOs se distinguen:

- **Error local de truncamiento** τ_i : error cometido en **un único paso**, suponiendo que se parte del valor exacto $y(t_i)$.
- **Error global** $e_i = y(t_i) - y_i$: error **acumulado** tras i pasos, cuando cada paso parte de la aproximación del paso anterior.
- El error global es, en general, **un orden inferior** al local: los errores se acumulan a lo largo de la trayectoria.

Error local de truncamiento

Supongamos $y \in C^2[a, b]$. Desarrollo de Taylor:

$$y(t_{i+1}) = y(t_i) + h y'(t_i) + \frac{h^2}{2} y''(\xi_i), \quad \xi_i \in (t_i, t_{i+1}).$$

Como $y'(t_i) = f(t_i, y(t_i))$:

$$\tau_i := y(t_{i+1}) - [y(t_i) + h f(t_i, y(t_i))] = \frac{h^2}{2} y''(\xi_i).$$

Si $|y''(t)| \leq M$ en $[a, b]$:

$$|\tau_i| \leq \frac{M}{2} h^2 = O(h^2).$$

Cota del error global

Teorema (Estimación del error del método de Euler)

Supongamos que $f(t, y)$ es continua y satisface una condición de Lipschitz con constante L en $[a, b] \times \mathbb{R}$, y que $|y''(t)| \leq M$ para todo $t \in [a, b]$, donde $y(t)$ es la solución única del PVI. Sean $y_0, y_1, \dots, y_N$ las aproximaciones obtenidas por el método de Euler. Entonces, para cada $i = 0, 1, \dots, N - 1$:

$$|y(t_{i+1}) - y_{i+1}| \leq \frac{h M}{2L} \left[e^{L(t_{i+1}-a)} - 1 \right].$$

Demostración: Computational Mathematics (Dimitrios Mitsotakis) — Teorema 8.3

Convergencia lineal

De la cota anterior:

$$|y(t_i) - y_i| \leq C h, \quad C = \frac{M}{2L} \left[e^{L(b-a)} - 1 \right].$$

- El método de Euler **converge linealmente** en h : al dividir h entre 10, el error se divide aproximadamente entre 10.
- **Observaciones:**
 - El error local es $O(h^2)$ pero el global es $O(h)$: se pierde un orden por la **acumulación** a lo largo de $N \sim 1/h$ pasos.
 - La cota crece **exponencialmente** con $b - a$ si L es grande: los problemas rígidos requerirán técnicas específicas.

Limitaciones de la convergencia lineal

- El método de Euler es útil por su simplicidad, pero su convergencia lineal es **poco eficiente**: reducir el error a la mitad exige duplicar el número de pasos.
- Estrategias para mejorar el orden:
 - **Euler mejorado (Heun)**: método explícito de orden $O(h^2)$ (se verá a continuación).
 - Familias más generales de métodos de orden superior, como los **Runge-Kutta** (no se desarrollan en este curso).
 - **Paso adaptativo**: ajuste dinámico de h según una estimación local del error.

Euler Implícito

Derivación: diferencia hacia atrás

- Evaluamos ahora la EDO en el nodo t_{i+1} :

$$y'(t_{i+1}) = f(t_{i+1}, y(t_{i+1})).$$

- Aproximamos $y'(t_{i+1})$ mediante la **diferencia hacia atrás**:

$$y'(t_{i+1}) \approx \frac{y(t_{i+1}) - y(t_i)}{h}.$$

- Se obtiene el método de **Euler implícito** (*backward Euler*):

$$y_{i+1} = y_i + h f(t_{i+1}, y_{i+1})$$

Carácter implícito del método

- Si f es **no lineal** en y , la ecuación

$$y_{i+1} = y_i + h f(t_{i+1}, y_{i+1})$$

no se puede resolver despejando y_{i+1} en forma cerrada.

- En cada paso es necesario resolver una **ecuación no lineal** en y_{i+1} :
 - Se trata de una ecuación de **punto fijo** $y_{i+1} = F(y_{i+1})$ con $F(z) = y_i + h f(t_{i+1}, z)$.
 - Son aplicables los métodos del Tema 2 (iteración de punto fijo, Newton).
- Cada paso tiene mayor coste que el del Euler explícito, a cambio de mejores propiedades de **estabilidad** (sección final).

Caso lineal: $y' = y$

Para $f(t, y) = y$ el método de Euler implícito produce:

$$y_{i+1} = y_i + h y_{i+1},$$

que admite resolución explícita:

$$y_{i+1} = \frac{1}{1 - h} y_i.$$

- Para EDOs **lineales en y** el Euler implícito conserva el coste del explícito.
- En problemas no lineales debe afrontarse el coste de resolver una ecuación no lineal en cada paso.

Orden de convergencia

- La derivación es análoga a la del Euler explícito, reemplazando la diferencia hacia adelante por la diferencia hacia atrás.
- **Error local:** $O(h^2)$. **Error global:** $O(h)$.
- El método implícito posee el **mismo orden de convergencia** que el explícito.
- Observación: el error local de truncamiento es $-\frac{h^2}{2}y''(\xi_i)$; coincide en módulo con el del Euler explícito y difiere en el signo.
- La elección del método implícito se justifica por razones de **estabilidad** frente a problemas *stiff*, no por su orden: los métodos implícitos admiten pasos h considerablemente mayores sin inestabilidades.

Euler Mejorado (Heun)

Motivación

- El método de Euler tiene convergencia **lineal** $O(h)$, insuficiente en muchas aplicaciones.
- Se busca un método de orden **superior** que siga siendo **explícito**.
- **Idea de Heun** (1859–1929):
 - i. Realizar una predicción de y_{i+1} con el método de Euler (pendiente en t_i).
 - ii. Utilizar esa predicción para estimar la pendiente en t_{i+1} .
 - iii. Promediar ambas pendientes para obtener el paso definitivo.

Fórmula del método

Método de Euler mejorado (Heun):

$$\tilde{y}_{i+1} = y_i + h f(t_i, y_i) \quad (\text{predicción})$$

$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, \tilde{y}_{i+1})] \quad (\text{corrección})$$

- **Fase predictora:** aplicación del Euler explícito.
- **Fase correctora:** media de pendientes en t_i y t_{i+1} .
- Coste: dos evaluaciones de f por paso (frente a una en Euler).

Derivación vía Taylor

Partiendo de la EDO $y' = f(t, y)$ y derivando respecto a t :

$$y'' = \frac{d}{dt} f(t, y(t)) = f_t(t, y) + f_y(t, y) \cdot f(t, y).$$

Desarrollo de Taylor hasta orden 3:

$$y(t + h) = y(t) + h f + \frac{h^2}{2} (f_t + f_y f) + O(h^3).$$

Reagrupando y utilizando el desarrollo de Taylor **multivariable** de $f(t + h, y + h f)$:

$$y(t + h) = y(t) + \frac{h}{2} f(t, y) + \frac{h}{2} f(t + h, y + h f) + O(h^3).$$

Derivación vía Taylor (cont.)

Identificando $\tilde{y}_{i+1} = y_i + h f(t_i, y_i)$ en la expresión anterior se recupera la fórmula del método:

$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, \tilde{y}_{i+1})].$$

- **Error local:** $O(h^3)$ (un orden por encima del de Euler).
- **Error global:** $O(h^2)$ (un orden por encima del de Euler).
- Al dividir h entre 10, el error se divide aproximadamente entre 100.

Heun como método de Runge-Kutta

Definiendo las **etapas intermedias**

$$k_1 = f(t_i, y_i), \quad k_2 = f(t_i + h, y_i + h k_1),$$

el método de Heun adopta la forma:

$$y_{i+1} = y_i + h \left(\frac{1}{2} k_1 + \frac{1}{2} k_2 \right).$$

- Heun es el miembro más simple de la familia de **métodos de Runge-Kutta (RK)**, cuya forma general es

$$y_{i+1} = y_i + h \sum_{j=1}^s b_j k_j, \quad k_j = f \left(t_i + c_j h, y_i + h \sum_{l=1}^{j-1} a_{jl} k_l \right).$$

Implementación en Python

```
def heun(t, f, y0):  
    n = len(t)  
    y = np.zeros(n)  
    h = (t[-1] - t[0]) / (n - 1)  
    y[0] = y0  
    for i in range(n - 1):  
        k1 = f(t[i], y[i])  
        k2 = f(t[i] + h, y[i] + h * k1)  
        y[i+1] = y[i] + 0.5 * h * (k1 + k2)  
    return y
```

- Estructura análoga a `euler` con dos evaluaciones de f por paso.
- Con $N = 10$ en el problema $y' = y$, $y(0) = 1$, $t \in [0, 2]$, las curvas exacta y numérica resultan prácticamente indistinguibles gráficamente.

Comparación de métodos

Método	Coste por paso	Error local	Error global
Euler explícito	1 evaluación de f	$O(h^2)$	$O(h)$
Euler implícito	1 evaluación + resolución de $F(z) = z$	$O(h^2)$	$O(h)$
Heun	2 evaluaciones de f	$O(h^3)$	$O(h^2)$

- Los métodos de orden superior resultan más eficientes en la práctica: admiten pasos h mayores para alcanzar una precisión dada.
- El coste total depende del par (precisión requerida, N), no sólo del coste por paso.

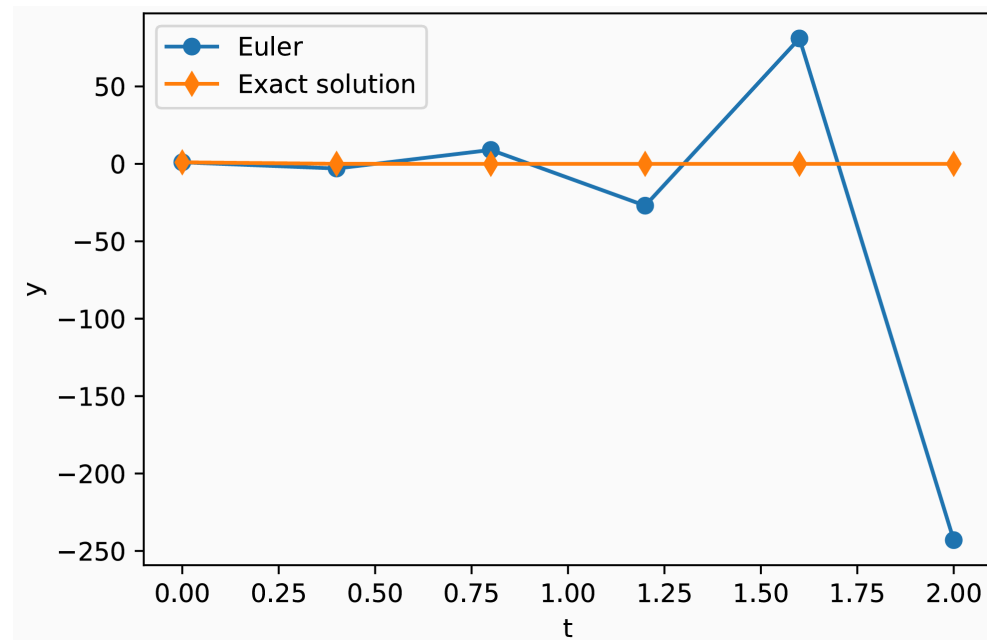
Estabilidad

Elección del paso

- Interesa tomar h **lo mayor posible** (menor coste, menor acumulación de redondeo).
- Sin embargo, con h demasiado grande el método puede producir resultados sin sentido físico, pese a ser convergente cuando $h \rightarrow 0$.
- **Cuestión:** ¿existe un **umbral superior** para h por encima del cual el método se vuelve inutilizable?

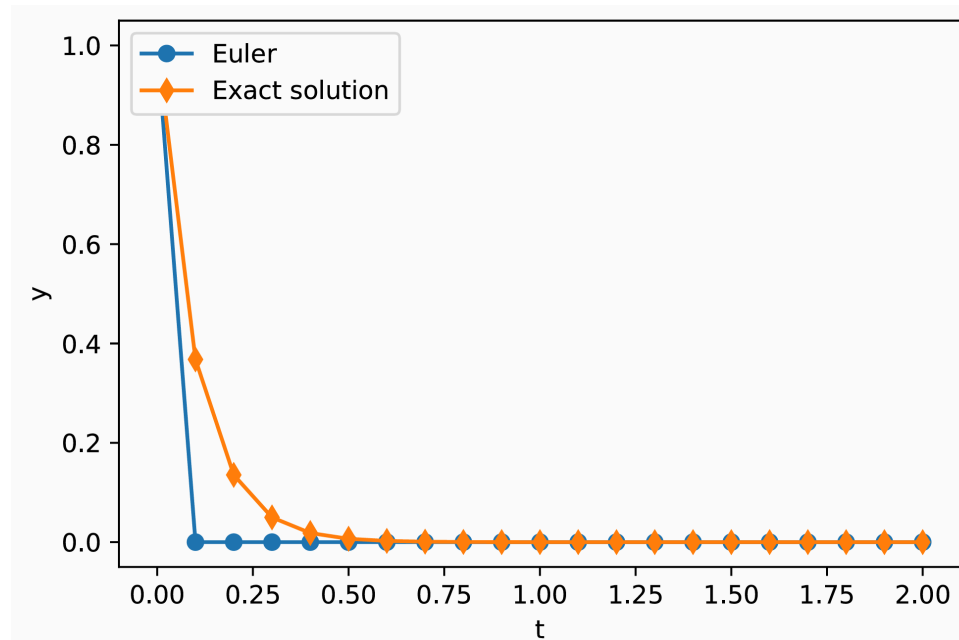
Ejemplo: $y' = \lambda y$ con $\lambda = -10$, paso grande

- Problema test: $y'(t) = \lambda y(t)$, $\lambda < 0$, $y(0) = 1$.
- Solución exacta: $y(t) = e^{\lambda t} \rightarrow 0$ cuando $t \rightarrow \infty$.



Con $h = 0.4$ el método de Euler **diverge** (oscila con amplitud creciente) mientras la solución exacta decae suavemente a cero.

Ejemplo: mismo problema con paso pequeño



Con $h = 0.1$ la aproximación reproduce correctamente el decaimiento. Queda por determinar el **umbral** que separa ambos comportamientos.

Análisis para el problema test $y' = \lambda y$

Aplicando el método de Euler explícito:

$$y_{n+1} = y_n + h \lambda y_n = (1 + h\lambda) y_n.$$

Iterando desde $y_0 = 1$:

$$y_n = (1 + h\lambda)^n.$$

Por tanto:

$$|y_n| \rightarrow 0 \iff |1 + h\lambda| < 1.$$

Restricción de estabilidad: $h < 2/|\lambda|$

Para $\lambda < 0$ y $h > 0$:

$$|1 + h\lambda| < 1 \iff -2 < h\lambda < 0 \iff h < -\frac{2}{\lambda} = \frac{2}{|\lambda|}.$$

- Con $\lambda = -10$: $h < 0.2$. Esto justifica rigurosamente el comportamiento observado en los ejemplos anteriores ($h = 0.4$ inestable, $h = 0.1$ estable).
- Si λ es muy negativo (p. ej. $\lambda = -10^6$): $h < 2 \times 10^{-6}$. Se requieren 10^6 pasos para integrar en $[0, 1]$, aunque la solución exacta decaiga en una escala de 10^{-6} y sea prácticamente nula en el resto del intervalo.
- Los problemas con **escalas temporales muy dispares** se denominan **rígidos** (*stiff*) y resultan muy exigentes para los métodos explícitos.

Euler implícito: A-estabilidad

Aplicando el método de Euler implícito al problema test:

$$y_{n+1} = y_n + h\lambda y_{n+1} \implies y_{n+1} = \frac{y_n}{1 - h\lambda}.$$

Iterando:

$$y_n = \frac{1}{(1 - h\lambda)^n}.$$

Para $\lambda < 0$ y $h > 0$ se tiene $1 - h\lambda > 1$, luego $|y_n| \rightarrow 0$ **para cualquier** $h > 0$.

- El Euler implícito es **incondicionalmente estable** en este problema: se dice que es **A-estable**.
- En general, los métodos implícitos suelen disfrutar de A-estabilidad: admiten **pasos h mayores** a cambio de un coste superior por paso.

Observaciones

- La **estabilidad** es una propiedad del **par (método, problema)**: un método puede ser estable en un problema y no en otro.
- Criterios prácticos:
 - Problemas no rígidos: un método explícito (Heun, o métodos Runge-Kutta de orden superior no estudiados aquí) es generalmente más eficiente.
 - Problemas rígidos: es imprescindible un método **implícito** (Euler implícito y sus variantes de orden superior).
- Los resolvedores profesionales (`scipy.integrate.solve_ivp` , `LSODA` , `SUNDIALS` , ...) combinan métodos explícitos e implícitos con **paso adaptativo** y detectan automáticamente la rigidez del problema.